Module 5: Reviewing and Testing AI Code

From plausible output to verified behavior

Module 5: Reviewing and Testing AI Code

Primary sources: Osmani, Chapter 5; Thomas & Hunt, Test to Code, Property-Based Testing, and Pragmatic Starter Kit.

Learning Outcomes

- Analyze AI-generated code for correctness and quality.
- Identify common failure patterns.
- Distinguish superficial correctness from true correctness.
- Apply testing strategies to validate outputs.

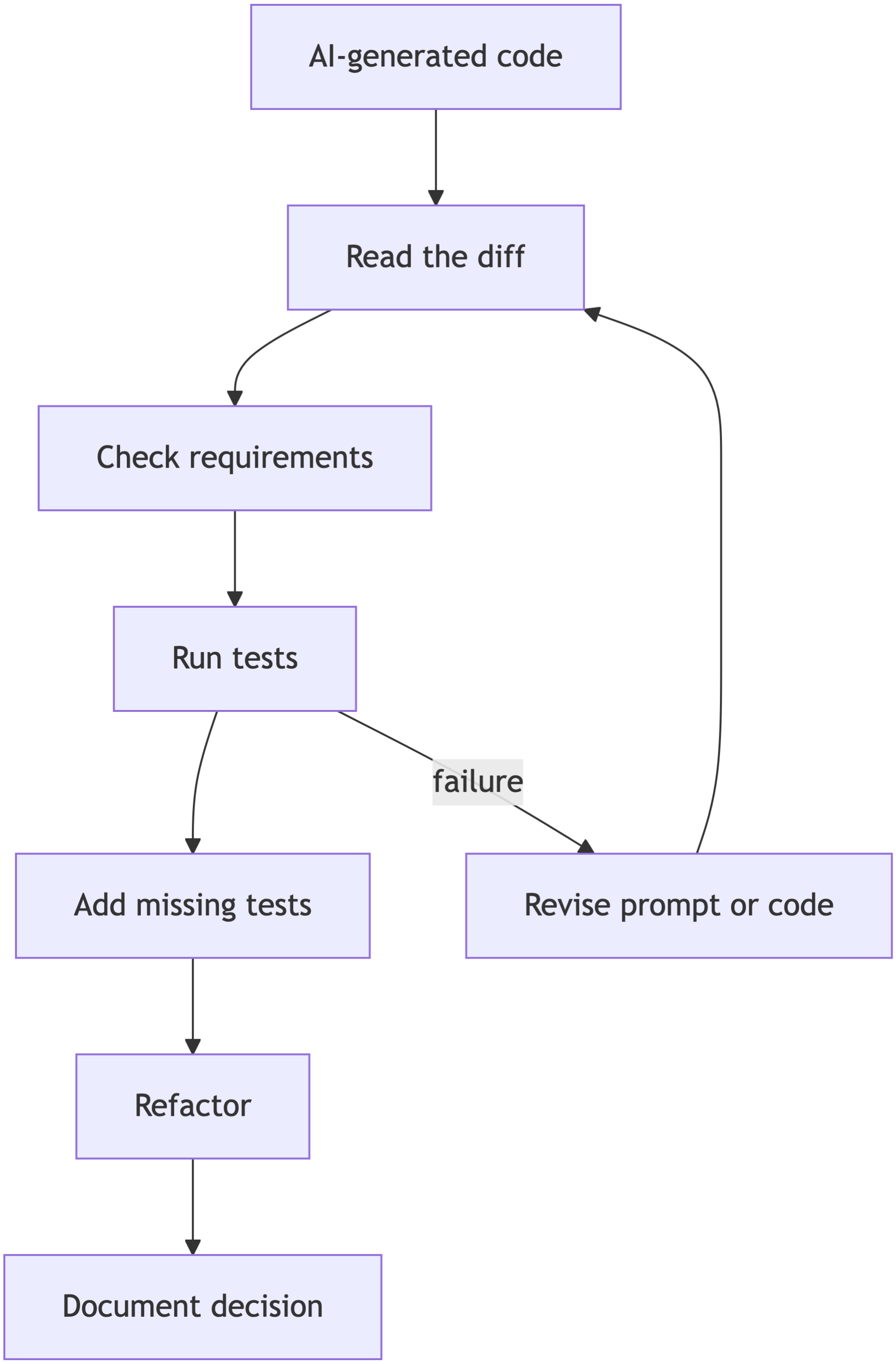
Review, Refine, Own

Osmani's core message for generated code: understanding is not optional.

You own:

- What was generated.
- What was accepted.
- What was changed.
- What was verified.
- What risk remains.

Review Pipeline



Superficial Correctness

Code may appear correct because it:

- Matches naming conventions.
- Compiles.
- Handles the example input.
- Uses familiar libraries.
- Includes comments that sound confident.

None of these prove behavior.

Review Checklist

- Does it implement the stated requirement?
- What assumptions did it add?
- What edge cases are missing?
- Is error handling intentional?
- Is the design testable?
- Does it expose secrets or unsafe behavior?
- Are dependencies appropriate?

Tests as Design

Thomas and Hunt argue testing is part of design, not an afterthought.

For AI-assisted work:

- Ask for tests before accepting implementation.
- Review tests as critically as code.
- Avoid tests that merely mirror the implementation.
- Add regression tests for AI mistakes.

Test Pyramid for AI Code

- Unit tests: fast checks of isolated behavior.
- Integration tests: verify components work together.
- End-to-end tests: validate user-visible workflow.
- Safety tests: verify refusal, bounds, permissions, and secrets handling.

Property Thinking

Property-based testing asks what must always be true.

Examples for Java command parsing:

- Parsing then serializing preserves arguments.
- Invalid commands never execute tools.
- Sandbox paths never escape the workspace.
- Empty input always fails safely.

Lab 3 Development Connection

As Lab 3 begins, implementation must remain inspectable.

Good evidence includes:

- Tool contracts.
- Sandbox restrictions.
- Tests proving restrictions.
- Sample runs.
- Copilot decisions and corrections.

Practice Activity

Give Copilot an intentionally incomplete implementation and ask it to review.

Then do a human review and compare:

- What did Copilot catch?
- What did it miss?
- Which issues require domain judgment?

Takeaways

- AI-generated code needs adversarial review.
- Tests are engineering evidence.
- Owning code means understanding failure behavior.